Floating-Point Divider Implementation In Verilog Using Newton-Raphson's Approximation

Project Proposal By Group 14 - stage 2

EDA playground link

Architecture overview:

The design follows a controller-datapath partition:

- **Control unit:** a finite state machine (FSM) sequences the computation: operand unpacking, initial estimate, the iterative refinement loop, the final multiplication and result packing. A counter bounds the number of iterations.
- **Datapath:** a 32-bit IEEE-754 single-precision multiplier and a 32-bit floating point subtractor, reused across iterations and controlled by the FSM.

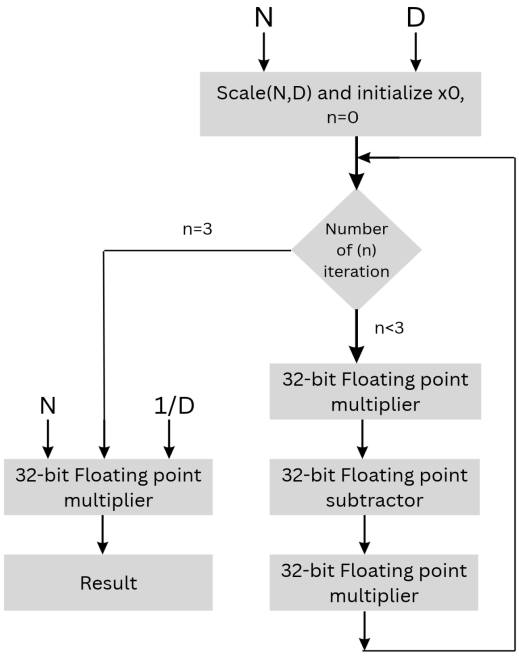


Fig: A Flowchart of the 32-bit floating point division using Newton-Raphson method.

We use three operation modules in our project:

1. **Multiplier Module**

The inputs given to the floating point multiplier are A[31-0] and B[31-0]. The multiplier module for IEEE-754 floating point is divided into three parts:

A. Mantissa Unit

In the mantissa unit, for calculation of mantissa a 24x24 bit Vedic multiplier is used efficiently for higher throughput and computation. The lower bits of inputs, m1 (A[22-0]) and m2 (B[22-0]) are given to the 24-bit Vedic multiplier which produces 24-bit normalized output and should have leading one as their MSB

B. Exponent Unit

In the Exponent unit, the exponent calculation is done by using ripple carry adder. The exponent is computed by providing inputs e1 (A[30-23]) and e2 (B[30 – 23]) to the 8-bit ripple carry adder unit and the result is biased to 127. The overflow and underflow cases are carefully handled.

C. Sign Unit

In the sign unit, the sign bit is computed by xoring the 31st bit of inputs, s1 (A[31]) and s2 (B[31]) of floating point inputs. The output of the xor gate represents the sign of the floating point multiplier.

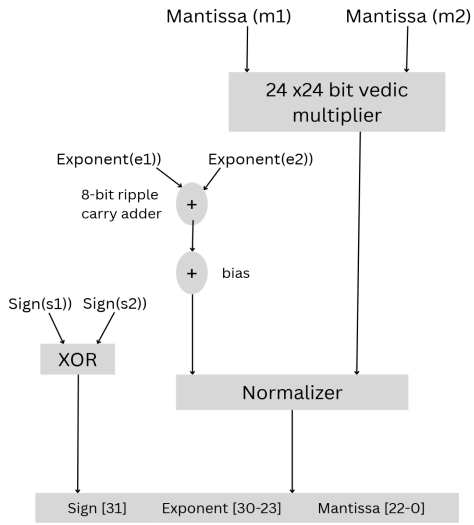


Fig: The proposed architecture for 32-bit floating point multiplier

2. Subtractor module

In the subtractor module, X[31-0] and Y[31-0] are given as inputs to the floating point subtractor. The sign (s), exponent (e) and mantissa (m) are represented in IEEE 754 format. 32-bit floating point subtraction operation is done in a stepwise manner as explained further. First of all, the floating point numbers are unpacked. After unpacking, the sign, exponent and mantissa are identified for performing the subtraction operation. Next, the exponent is equalized for performing alignment and normalization of the mantissa part. If neither of the operands are infinite, then the relation between e1 (X[30-23]) and e2 (Y[30-23]) is determined by comparing the mantissa m1 and m2. The mantissa is shifted right until the exponent becomes equal, i.e. e1 (X[30-23]) = e2 (Y[30- 23]). After alignment and normalization, the mantissa m1 (X[23-0]) and m2 (Y[23-0]) are subtracted. After that, the mantissa values are rounded off. Finally, the sign, exponent and mantissa parts are concatenated.

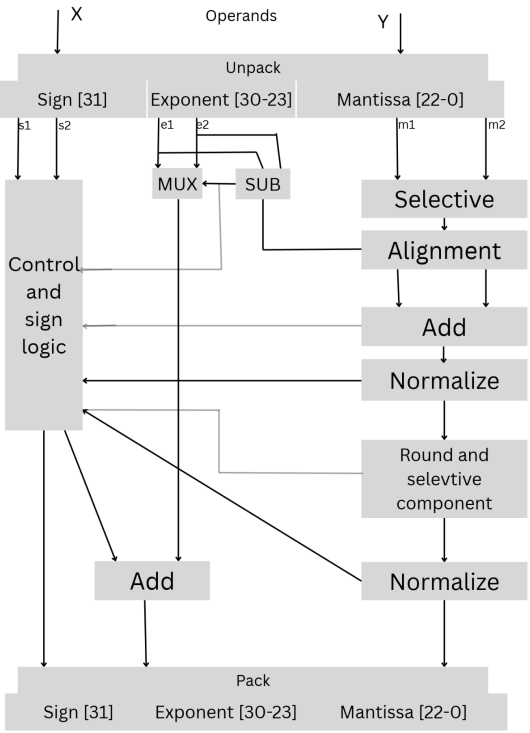


Fig: The architecture of 32-bit floating point substractor.

These modules are used to compute Newton-Raphson reciprocal and compute the final multiplication.

Design Methodology

1. Floating-point arithmetic units

- **Multiplier:** the sign is the XOR of the operand signs, the exponents are added and the bias subtracted, and the 24×24-bit significands are multiplied. The 48-bit product is then normalized, rounded, and the exponent adjusted.
- **Subtractor:** the operand exponents are compared and the smaller significand is right-shifted to align it. The significands are then subtracted, the result is normalized by a leading-zero shift with a matching exponent correction, and it is rounded and repacked.

2. Current implementation status

The top-level FSM and the interfaces between it and the multiplier and subtractor modules are complete. This validates the control flow, the data movement between states, and the iteration structure of the algorithm. The internal logic of the multiplier and subtractor is not yet implemented; for now they are modeled behaviorally with placeholder operations. The IEEE-754 unpacking procedure and the multiplication and subtraction algorithms have been studied and specified, and the next phase is to translate them into synthesizable RTL.

3. Planned next steps.

1. Implement the floating-point multiplier and subtractor as separate modules, with rounding and normalization.
2. Replace the placeholder operations in the top-level model with these modules.
3. Verify each module in its own testbench against reference values then verify the full divider and measure the error after each iteration.

Problems encountered:

1. Finding the most efficient multiplication method for mantissa multiplication: array multiplication ripples the carry to all digits and is inefficient; vedic multiplication is the fastest one for our implementation.
2. IEEE-754 normalisation and rounding off
3. Handling overflow and underflow

Workload Division:

1. Gurnoor, Keerat: Writing the code for the model as well as the placeholder modules + creating github repository for the project.
2. Monish: Researching for most efficient multiplier and subtractor modules + creating block diagrams for the same
3. Divya and Mansi: IEEE floating point unpacking, rounding and normalization units + creating block diagrams for the model design.

References:

1. From Baseline to High Throughput: Architecture and Resource Co-Optimization of a Verilog-Based Newton–Raphson Divider on FPGAs - Wentao Ding
2. C. S. Aragula, B. Preethi and C. Ramesh, "Design and Implementation of an Efficient 32-Bit Fixed-Point Newton-Raphson Division-Based Reciprocal Computing Unit
3. Design and Synthesis of Single Precision Floating Point Division based on Newton-Raphson Algorithm on FPGA - Naginder Singh and Trailokya Nath Sasamal